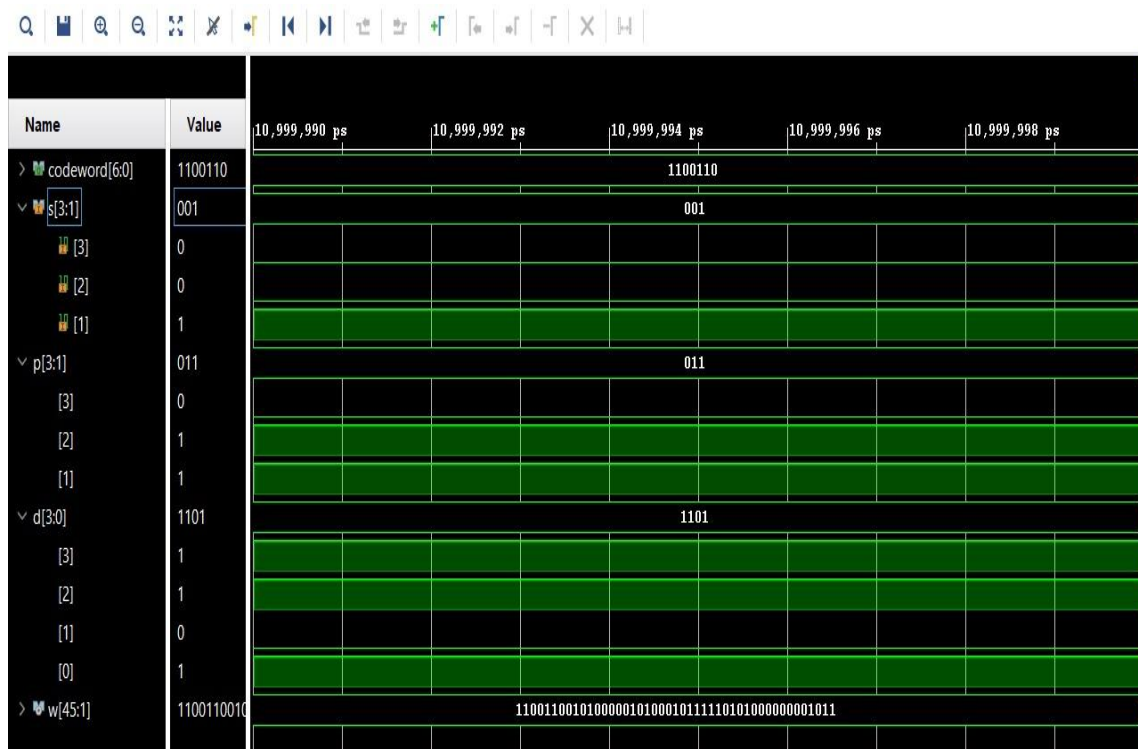


Existing Method-1

Hamming code Encoder and Decoder

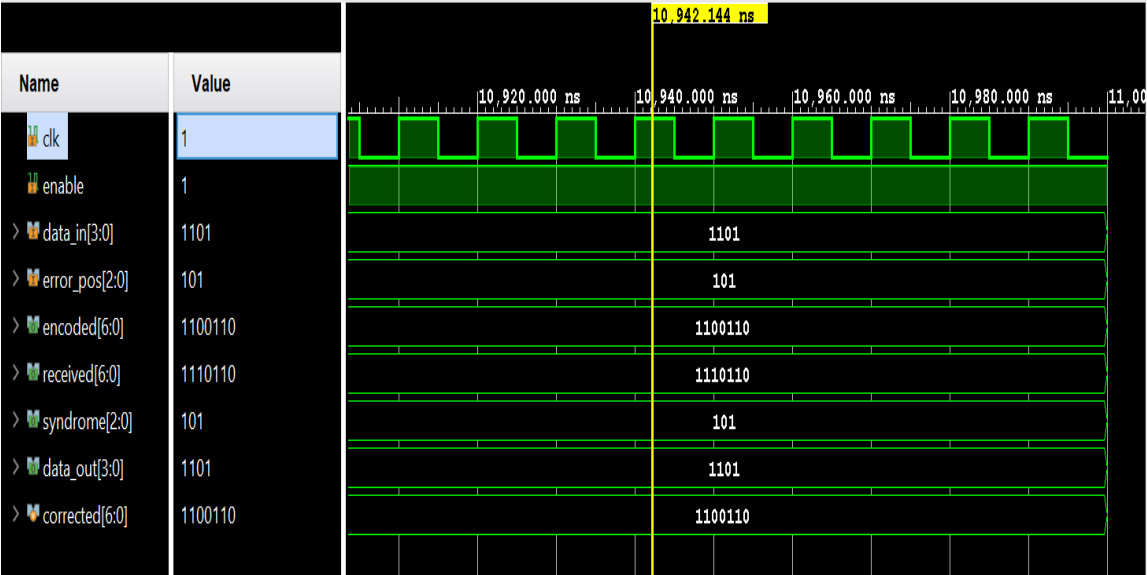


Waveforms of Error Correction Block



PROPOSED METHOD

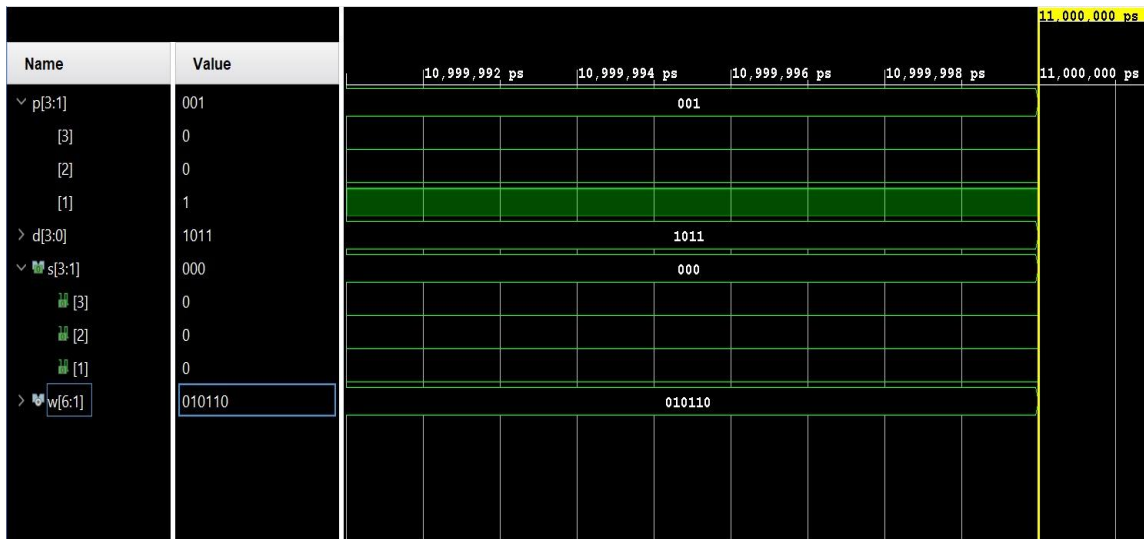
Simulation Waveform of Clock Gated Hamming Code Encoder and Decoder



Simulation Waveform of Proposed Reversible Logic Encoder



Simulation Waveform of Proposed Reversible Logic Decoder



Simulation Waveform of Error Detection and Correction Block



```

module fe(codeword,s,p,d);
input [3:0]d;
output [3:1]s;
inout [3:1]p;
output [6:0]codeword;
wire [9:1]w;
xor n1(w[1],d[0],d[1]);
xor n2(w[2],d[2],d[0]);
xor n3(w[3],d[1],d[2]);
xor n4(p[1],w[1],d[3]);
xor n5(p[2],w[2],d[3]);
xor n6(p[3],w[3],d[3]);
xor n7(w[4],1,d[0]);
xor n8(w[5],p[2],d[0]);
xor n9(w[6],d[1],p[3]);
xor n10(w[7],w[4],d[1]);

```

```

xor n11(w[8],d[2],w[5]);
xor n12(w[9],w[6],d[2]);
xor n13(s[1],w[7],d[3]);
xor n14(s[2],w[8],d[3]);
xor n15(s[3],w[9],d[3]);
assign codeword = {d[3],d[2],d[1],p[3],d[0],p[2],p[1]};
endmodule

```

Existing Correction Block

```

module fec(codeword,s,p,d);
output [6:0]codeword;
input [3:1]s;
inout [3:1]p;
inout [3:0]d;
wire [45:1]w;
//not gates
not n1(w[1],s[3]);
not n2(w[2],s[2]);
not n3(w[3],s[1]);
not n11(w[11],w[4]);
not n12(w[12],p[1]);
not n13(w[13],w[5]);
not n14(w[14],p[2]);
not n15(w[15],w[6]);
not n16(w[16],d[0]);
not n17(w[17],w[7]);
not n18(w[18],p[3]);
not n19(w[19],w[8]);
not n20(w[20],d[1]);
not n21(w[21],w[9]);
not n22(w[22],d[2]);
not n23(w[23],w[10]);
not n24(w[24],d[3]);
//and gate
and n4(w[4],w[1],w[2],s[1]);
and n5(w[5],w[1],s[2],w[3]);
and n6(w[6],w[1],s[2],s[3]);
and n7(w[7],s[3],w[2],w[3]);
and n8(w[8],s[3],w[2],s[1]);
and n9(w[9],s[3],s[2],w[3]);
and n10(w[10],s[3],s[2],s[1]);
//right
and n25(w[25],w[11],p[1]);
and n26(w[26],w[12],w[4]);
and n27(w[27],w[13],p[2]);
and n28(w[28],w[14],w[5]);
and n29(w[29],w[15],d[0]);
and n30(w[30],w[16],w[6]);
and n31(w[31],w[17],p[3]);
and n32(w[32],w[18],w[7]);

```

```

and n33(w[33],w[19],d[1]);
and n34(w[34],w[20],w[8]);
and n35(w[35],w[21],d[2]);
and n36(w[36],w[22],w[9]);
and n37(w[37],w[23],d[3]);
and n38(w[38],w[24],w[10]);
//or gates
or n39(w[39],w[25],w[26]);
or n40(w[40],w[27],w[28]);
or n41(w[41],w[29],w[30]);
or n42(w[42],w[31],w[32]);
or n43(w[43],w[33],w[34]);
or n44(w[44],w[35],w[36]);
or n45(w[45],w[37],w[38]);
assign codeword={w[45],w[44],w[43],w[42],w[40],w[41],w[39]};
endmodule

```

Clock Gating Technique

```

module cgt(codeword,s,p,clk,d);
input [3:0]d;
output [3:1]s;
inout [3:1]p;
input clk;
output [6:0]codeword;
wire [13:1]w;
//AND GATE
and n1(w[1],d[0],clk);
and n2(w[2],d[1],clk);
and n3(w[3],d[2],clk);
and n4(w[4],d[3],clk);
xor n5(w[5],w[1],w[2]);
xor n6(w[6],w[1],w[3]);
xor n7(w[7],w[2],w[3]);
xor n8(p[1],w[5],w[4]);
xor n9(p[2],w[6],w[4]);
xor n10(p[3],w[7],w[4]);
xor n11(w[8],p[1],w[1]);
xor n12(w[9],p[2],w[1]);
xor n13(w[10],w[2],p[3]);
xor n14(w[11],w[2],w[8]);
xor n15(w[12],w[9],w[3]);
xor n16(w[13],w[10],w[3]);
xor n17(s[1],w[11],w[4]);
xor n18(s[2],w[12],w[4]);
xor n19(s[3],w[13],w[4]);
assign codeword = {d[3],d[2],d[1],p[3],d[0],p[2],p[1]};
endmodule

```

Proposed Enocder and Decoder

```

module fp1(codeword,p,s,d);

```

```

output [6:0]codeword;
input [3:0]d;
inout [3:1]p;
output [3:1]s;
wire [11:1]w;
//encoder
xor n1(w[1],d[0],d[1]);
xor n2(w[2],d[0],d[2]);
xor n3(p[1],d[3],w[1]);
xor n4(p[2],d[3],w[2]);
xor n5(w[3],d[1],d[2]);
xor n6(w[4],w[3],d[1]);
xor n7(w[5],d[3],d[1]);
xor n8(p[3],w[4],w[5]);
//decoder
xor n9(w[6],d[0],p[1]);
xor n10(w[7],p[2],d[0]);
xor n11(w[8],w[6],d[3]);
xor n12(w[9],w[7],d[3]);
xor n13(w[10],w[9],d[1]);
xor n14(w[11],w[10],d[3]);
xor n15(s[3],w[11],p[3]);
xor n16(s[1],w[8],d[1]);
xor n17(s[2],d[2],w[9]);
assign codeword={d[3],d[2],d[1],p[3],d[0],p[2],p[1]};
endmodule

```

Proposed Correction

```

module fc(p,d,s);
inout [3:1]p;
inout [3:0]d;
input [3:1]s;
wire [21:1]w;
//xor gates
xor n1(w[1],s[1],0);
xor n2(w[2],s[2],s[1]);
xor n3(w[3],1,s[1]);
xor n4(w[4],1,s[2]);
xor n5(w[5],0,w[3]);
xor n6(w[6],w[4],w[3]);
xor n7(w[7],0,s[3]);
xor n8(w[8],s[3],w[1]);
xor n9(w[9],0,s[3]);
xor n10(w[10],s[3],w[2]);
xor n11(w[11],0,s[2]);
xor n12(w[12],s[2],0);
xor n13(w[13],w[6],s[2]);
xor n14(w[14],d[3],w[7]);
xor n15(w[15],d[2],w[8]);
xor n16(w[16],d[0],w[9]);

```

```
xor n17(w[17],p[2],w[10]);  
xor n18(w[18],p[1],w[11]);  
xor n19(w[19],d[1],w[12]);  
xor n20(w[20],p[3],w[13]);  
endmodule
```